Computação Distribuída com RPC (gRPC)

Prof Alcides/Prof Mario

Laboratório (gRPC + Python)

Resumo

O objetivo da aula é compreender a comunicação entre cliente e servidor por meio do gRPC, implementando em Python um sistema de gerenciamento de notas. O contrato .proto define os métodos e dados compartilhados entre as aplicações.

A atividade compara chamadas unárias, nas quais uma requisição gera uma resposta, com o server streaming, no qual uma única solicitação permite ao servidor enviar várias notas progressivamente. Em uma API REST com HTTP/1.1, normalmente todas as notas seriam reunidas em um JSON e enviadas em uma única resposta. Embora o HTTP/1.1 também permita transmissão em partes (chunked), o gRPC oferece suporte direto a fluxos de mensagens estruturadas.

O streaming possui finalidade didática neste laboratório, mas é útil em sistemas com muitos registros ou processamento demorado, pois permite ao cliente começar a tratar os primeiros resultados antes de receber todos os dados.

Objetivos de Aprendizagem

1. Definir um serviço gRPC usando Protocol Buffers (Protobuf). (20%)
2. Implementar um servidor e um cliente gRPC em Python. (40%)
3. Implementar uma chamada RPC com streaming de dados (Server Streaming). (40%)

No gRPC, uma chamada pode ser:

- Unary RPC: cliente manda 1 requisição, servidor devolve 1 resposta (caso normal, como AdicionarNota).
  rpc EnviarMensagem(Mensagem) returns (Resposta);
- Server Streaming RPC: cliente manda 1 requisição, mas o servidor devolve várias respostas em sequência, como se fosse um "fluxo" (stream).
  rpc ReceberMensagens(Filtro) returns (stream Mensagem);
- Client Streaming RPC: cliente manda várias requisições, servidor devolve 1 resposta.
  rpc EnviarMensagens(stream Mensagem) returns (Resumo);
- Bidirectional Streaming RPC: os dois lados trocam várias mensagens ao mesmo tempo.
  rpc Chat(stream Mensagem) returns (stream Mensagem);

Passo 1. O Contrato – notas.proto

Este é o coração do projeto. O arquivo .proto será a fonte da verdade para ambas as implementações:

```
syntax = "proto3";

package gerencia_notas;

// Estrutura para uma nota individual
message Nota {
    string ra = 1;
    string cod_disciplina = 2;
    int32 ano = 3;
    int32 semestre = 4;
    float nota = 5;
}

// Resposta para operações de consulta de uma única nota
message ConsultaNotaResponse {
    bool sucesso = 1;
    Nota nota = 2;
    string msg_erro = 3;
}

// Requisição para adicionar ou consultar uma nota
message AlunoDisciplinaRequest {
    string ra = 1;
    string cod_disciplina = 2;
}

// Requisição para adicionar uma nota completa
message AdicionaNotaRequest {
    string ra = 1;
    string cod_disciplina = 2;
    int32 ano = 3;
    int32 semestre = 4;
    float nota = 5;
}

// Resposta genérica de status (sucesso/falha)
message StatusResponse {
    bool sucesso = 1;
    string msg = 2;
}

// Requisição para calcular média, identificando apenas o aluno
message AlunoRequest {
    string ra = 1;
}

// Resposta com o valor da média
message MediaResponse {
    bool sucesso = 1;
    float media = 2;
    string msg_erro = 3;
}

// Definição do serviço
service GerenciadorNotas {
    // RPCs Unários (requisição/resposta simples)
    rpc AdicionarNota(AdicionaNotaRequest) returns (StatusResponse) {}
    rpc AlterarNota(AdicionaNotaRequest) returns (StatusResponse) {}
    rpc ConsultarNota(AlunoDisciplinaRequest) returns (ConsultaNotaResponse) {}
    rpc CalcularMedia(AlunoRequest) returns (MediaResponse) {}

    // ***** RPC Desafio *****
    // Server Streaming: cliente pede as notas de um aluno, servidor envia um stream
de notas
    rpc ListarNotasAluno(AlunoRequest) returns (stream Nota) {}
}
```

Parte 2. Implementação em Python

- Por que começar com Python? Isso nos permitirá focar nos conceitos do gRPC com mínimo de sobrecarga de compilação e gerenciamento de dependências, obtendo um resultado funcional rapidamente.

2.1. Preparação do Ambiente Python

```
:~$ # Criar e ativar ambiente virtual

:~$ python -m venv venv

:~$ #Linux

:~$ source venv/bin/activate

:~$ #Windows

:~$ venv\Scripts\activate


:~$ # Instalar bibliotecas

:~$ pip install grpcio grpcio-tools
```

2.2. Gerar Código a partir do .proto

```
:~$ python -m grpc_tools.protoc -I. --python_out=. --pyi_out=.  --
grpc_python_out=. notas.proto
```

OBS: Isso gera os arquivos: notas_pb2.py, notas_pb2.pyi, notas_pb2_grpc.py.

2.3. Servidor (servidor.py – esqueleto)

Agora devemos criar uma classe que herda de GerenciadorNotasServicer e implementar os métodos. O "banco de dados" pode ser um simples dicionário em memória.

```python
# servidor.py (esqueleto)
import grpc
from concurrent import futures
import notas_pb2
import notas_pb2_grpc

# Armazenamento em memória (simulando um banco de dados)
db_notas = {} # Ex: { "123_DISC1": Nota(...), "123_DISC2": Nota(...) }

class GerenciadorNotasServicer(notas_pb2_grpc.GerenciadorNotasServicer):

    def AdicionarNota(self, request, context):
        print(f"Adicionando nota para RA {request.ra} na disciplina
{request.cod_disciplina}")
        chave = f"{request.ra}_{request.cod_disciplina}"
        if chave in db_notas:
            return notas_pb2.StatusResponse(sucesso=False, msg="Nota já existe para
este aluno/disciplina. Use AlterarNota.")

        # Lógica para criar e armazenar o objeto Nota
        # ...
        db_notas[chave] = ...

        return notas_pb2.StatusResponse(sucesso=True, msg="Nota adicionada com
sucesso!")
```

```
    # ... Implementar os outros métodos (AlterarNota, ConsultarNota, CalcularMedia)
...

    # --- IMPLEMENTAÇÃO DO DESAFIO ---
    def ListarNotasAluno(self, request, context):
        """
        DESAFIO: Este método deve usar Server-Side Streaming.
        Ele deve buscar todas as notas associadas ao RA do aluno no 'banco de dados'
        e enviá-las uma a uma para o cliente usando a palavra-chave 'yield'.
        """
        print(f"Listando notas para o RA {request.ra} (NÃO IMPLEMENTADO)")

        # TODO: Implementar a lógica do desafio aqui.
        # Dica: itere pelo seu dicionário `db_notas` e
        # use `yield` para cada nota encontrada.

        # Exemplo de como retornar um erro de "não implementado"
        context.set_code(grpc.StatusCode.UNIMPLEMENTED)
        context.set_details('Método não implementado!')
        return notas_pb2.Nota() # Precisa retornar um objeto do tipo
        #                         esperado, mesmo que vazio

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    notas_pb2_grpc.add_GerenciadorNotasServicer_to_server(GerenciadorNotasServicer(),
server)
    server.add_insecure_port('[::]:50051')
    server.start()
    print("Servidor gRPC rodando na porta 50051.")
    server.wait_for_termination()

if __name__ == '__main__':
    serve()
```

2.4. Cliente (cliente.py – esqueleto)

```
# cliente.py (esqueleto)
import grpc
import notas_pb2
import notas_pb2_grpc

def run():
    with grpc.insecure_channel('localhost:50051') as channel:
        stub = notas_pb2_grpc.GerenciadorNotasStub(channel)

        # Exemplo de chamada: Adicionar Nota
        response =
stub.AdicionarNota(notas_pb2.AdicionaNotaRequest(ra="123",cod_disciplina="CIC0001",
ano=2025, semestre=2, nota=9.5))
        print(f"Resposta do servidor: {response.msg}")

        # Exemplo de chamada: Calcular Média
        media_response = stub.CalcularMedia(notas_pb2.AlunoRequest(ra="123"))
        if media_response.sucesso:
            print(f"Média do aluno 123: {media_response.media}")
        else:
            print(f"Erro ao calcular média: {media_response.msg_erro}")

        # --- CHAMADA DO DESAFIO (STREAMING) ---
        # TODO: Descomente este bloco de código após implementar o desafio
        # no servidor.
        # O objetivo é iterar sobre o stream de respostas recebido do
        # servidor e imprimir cada nota.
        """
        print("\n--- Listando todas as notas do aluno 123 (DESAFIO) ---")
        try:
            notas_stream = stub.ListarNotasAluno(notas_pb2.AlunoRequest(ra="123"))
            for nota in notas_stream:
```

```
                print(f"  - Disciplina: {nota.cod_disciplina}, Nota: {nota.nota}")
        except grpc.RpcError as e:
            print(f"Erro ao chamar ListarNotasAluno: {e.details()}")
        """

if __name__ == '__main__':
    run()
```

Passe 3. Execução

```
:~$ # 1) Em um terminal: iniciar servidor

:~$ python servidor.py



:~$ # 2) Em outro terminal: executar cliente

:~$ python cliente.py
```

Passo 4. Entrega

O exercício deve ser feito em dupla e entregue no laboratório rodando.